

QUANDELA : Le modèle computationnel photonique discret (DV) et les fondements de Perceval

Karim El Houdaigui

Contents

1	Introduction générale	3
2	Les modes optiques et la base de Fock	3
2.1	Définition d'un mode	3
2.2	Espace de Hilbert global	3
3	Encodage de l'information : le dual-rail	3
3.1	Principe de l'encodage	3
3.2	Pourquoi ce choix ?	4
3.3	Exemple : deux qubits	4
4	Les opérateurs de création et d'annihilation	4
5	Les opérations optiques fondamentales	4
5.1	Le déphaseur P_ϕ	5
5.2	Le diviseur de faisceau B_θ	5
5.3	Cas deux photons incident : interférence de Hong–Ou–Mandel	6
6	Simulation dans Perceval	6
7	Résumé conceptuel	7
8	Extension : Polarisation, Plaques Biréfringentes et Diviseurs Polarisants	7
8.1	Modes de polarisation et espace de Hilbert	7
8.2	Plaques biréfringentes : transformations unitaires sur la polarisation	7
8.3	Diviseur de faisceau polarisant (PBS)	8
8.4	Applications et exemples	9
8.5	Remarques expérimentales et modélisation	9
8.6	Implémentation dans <i>Perceval</i>	10
8.7	Vérifications essentielles	10
9	Feedforward : contrôle adaptatif par la mesure	10
9.1	Limites des réseaux optiques linéaires et rôle du <i>feedforward</i>	10
9.1.1	Le besoin de non-linéarités pour le calcul universel	10
9.1.2	La mesure comme source de non-linéarité effective	11
9.1.3	Principe du <i>feedforward</i>	11

9.1.4	Exemple : la porte CNOT photonique	11
9.1.5	Rôle de la mesure et du <i>feedforward</i>	12
9.2	Implémentation dans Perceval	12
9.2.1	Exemple minimal en Python	12
9.2.2	Conditionnement du calcul	13
9.3	Architecture interne du <i>feedforward</i> dans Perceval	13
9.4	Schéma physique du feedforward (encodage dual-rail)	14
9.5	Synthèse	14
10	Structure interne de Perceval : du photon au processeur	14
10.1	Les États : BasicState et StateVector	15
10.2	Les Circuits : Circuit et les composants optiques	16
10.3	Les <i>Backends</i> : simulateurs de circuits photoniques	17
10.4	Les Processeurs : Processor	18
10.5	Vision d'ensemble	19
10.6	Exécution réelle sur QPU (Quandela Cloud)	19
10.7	Résumé : du simulateur au QPU réel	20
11	Architecture du QPU photonique	21
11.0.1	(a) Source de photons uniques — Single Photon Source (SPS)	21
11.0.2	(b) Le démultiplexeur temporel	21
11.0.3	(c) La puce photonique intégrée	22
11.0.4	(d) Vers l'intrication photonique	22
11.1	Algorithmes photoniques implémentés	22
11.2	Avantages structurels de la photonique	23
11.2.1	1. Fabrication industrielle	23
11.2.2	2. Compatibilité avec le HPC	23
11.2.3	3. Hybridation naturelle	23
11.2.4	4. Flux de calcul photonique	23
11.3	Conclusion : de la recherche à l'industrie	23

1 Introduction générale

L'informatique quantique photonique repose sur l'utilisation des **photons uniques** comme porteurs d'information. Ces photons peuvent être manipulés à l'aide de **circuits optiques linéaires**, composés de diviseurs de faisceaux, de déphaseurs, et de détecteurs.

Le modèle que nous étudions ici est celui à **variables discrètes (DV, Discrete Variable)**, où l'information est encodée dans le **nombre de photons** présents dans différents chemins optiques, appelés **modes**.

Ce formalisme est à la base du logiciel **Perceval**, développé par **Quandela**, pour la conception et la simulation de circuits photoniques quantiques.

2 Les modes optiques et la base de Fock

2.1 Définition d'un mode

Un **mode optique** est une voie indépendante dans laquelle un photon peut se propager :

- fibre optique,
- guide d'onde intégré,
- direction spatiale,
- polarisation (H/V).

Chaque mode peut contenir un certain nombre de photons. On décrit alors l'état d'un mode par un **état de Fock** :

$$|0\rangle : \text{aucun photon}, \quad |1\rangle : \text{un photon}, \quad |2\rangle : \text{deux photons}, \dots$$

2.2 Espace de Hilbert global

Pour un système de m modes, l'espace de Hilbert est :

$$\mathcal{H} = \text{span}\{|n_1, n_2, \dots, n_m\rangle\}$$

où n_i représente le nombre de photons dans le mode i .

3 Encodage de l'information : le dual-rail

3.1 Principe de l'encodage

Un **qubit photonique** est représenté par un seul photon réparti entre deux modes distincts :

$$|0\rangle_{\text{logique}} = |1, 0\rangle, \quad |1\rangle_{\text{logique}} = |0, 1\rangle.$$

Cet encodage est appelé **dual-rail** : un photon sur deux rails optiques forme un qubit logique.

3.2 Pourquoi ce choix ?

- **Facile à manipuler** : les circuits optiques linéaires travaillent naturellement sur des interférences entre chemins.
- **Robuste** : les pertes ou déphasages identiques sur les deux rails préservent la cohérence.
- **Universel** : toute transformation unitaire peut être réalisée par des combinaisons de déphaseurs et de diviseurs de faisceaux.

Ainsi, pour représenter n qubits, il faut $2n$ modes :

$$n_{\text{modes}} = 2 \times n_{\text{qubits}}.$$

3.3 Exemple : deux qubits

$$|00\rangle = |1, 0, 1, 0\rangle, \quad |01\rangle = |1, 0, 0, 1\rangle, \quad |10\rangle = |0, 1, 1, 0\rangle, \quad |11\rangle = |0, 1, 0, 1\rangle.$$

4 Les opérateurs de création et d'annihilation

L'état d'un mode est décrit à l'aide des **opérateurs bosoniques** :

$$\begin{cases} a^\dagger : \text{opérateur de création d'un photon,} \\ a : \text{opérateur d'annihilation d'un photon.} \end{cases}$$

Ces opérateurs agissent sur les **états de Fock** $|n\rangle$ selon les règles suivantes :

$$\begin{cases} a^\dagger |n\rangle = \sqrt{n+1} |n+1\rangle \\ a |n\rangle = \sqrt{n} |n-1\rangle \end{cases}$$

Dans un **circuit optique linéaire**, l'évolution du système est décrite par une **transformation unitaire** sur les opérateurs de création :

$$\boxed{a_{\text{out}}^\dagger = U a_{\text{in}}^\dagger}$$

où U est une **matrice unitaire** représentant l'ensemble du circuit optique.

5 Les opérations optiques fondamentales

Les circuits photoniques linéaires reposent sur deux composants passifs :

- Le déphaseur (Phase Shifter, PS),
- Le diviseur de faisceau (Beam Splitter, BS).

5.1 Le déphaseur P_ϕ

Définition – Déphaseur P_ϕ

Le **déphaseur** (ou *Phase Shifter*) est un composant optique qui applique une phase globale $e^{i\phi}$ au champ quantique d'un mode donné. Son action sur l'opérateur de création est :

$$a^\dagger \longrightarrow e^{i\phi} a^\dagger.$$

Cette opération est **unitaire** et ne modifie pas le nombre de photons présents dans le mode : elle agit uniquement sur la **phase relative** de l'état. Ainsi, un photon traversant un déphaseur conserve son amplitude mais acquiert une phase supplémentaire $e^{i\phi}$.

5.2 Le diviseur de faisceau B_θ

Définition – Diviseur de faisceau B_θ

Le **diviseur de faisceau** (ou *Beam Splitter*) est un composant optique à deux entrées et deux sorties qui mélange les amplitudes des champs quantiques de deux modes distincts. Son action est décrite par une transformation unitaire sur les opérateurs de création :

$$\begin{pmatrix} a_1^\dagger \\ a_2^\dagger \end{pmatrix}_{\text{out}} = \hat{U}_{BS}(\theta) \begin{pmatrix} a_1^\dagger \\ a_2^\dagger \end{pmatrix}_{\text{in}}, \quad \text{où } \hat{U}_{BS}(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & i \sin\left(\frac{\theta}{2}\right) \\ i \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}.$$

Paramètres physiques

Le paramètre θ détermine la proportion d'intensité transmise ou réfléchi :

$$\begin{cases} T = \cos^2\left(\frac{\theta}{2}\right) & \text{probabilité de transmission} \\ R = \sin^2\left(\frac{\theta}{2}\right) & \text{probabilité de réflexion} \end{cases} \quad \text{et} \quad \boxed{T + R = 1}$$

Le **diviseur de faisceau idéal** ne produit aucune perte : il ne fait que redistribuer la probabilité de présence du photon entre les deux modes.

Action sur les états

Sur les opérateurs de création, le mélange se traduit par :

$$\begin{cases} a_1^\dagger \longrightarrow \cos\left(\frac{\theta}{2}\right) a_1^\dagger + i \sin\left(\frac{\theta}{2}\right) a_2^\dagger, \\ a_2^\dagger \longrightarrow i \sin\left(\frac{\theta}{2}\right) a_1^\dagger + \cos\left(\frac{\theta}{2}\right) a_2^\dagger. \end{cases}$$

Ainsi, un photon incident sur le premier mode ressort dans une **superposition cohérente** entre les deux sorties.

Cas particulier : Beam Splitter 50/50

Pour un diviseur de faisceau équilibré ($\theta = \pi/2$), la matrice devient :

$$\hat{U}_{BS}\left(\frac{\pi}{2}\right) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & i \\ i & 1 \end{pmatrix}.$$

Ce cas particulier correspond à un partage égal des amplitudes : la probabilité de transmission et de réflexion vaut $1/2$. En optique quantique, ce composant agit de manière analogue à une **porte de Hadamard** appliquée à un qubit :

$$|1, 0\rangle \longrightarrow \frac{1}{\sqrt{2}}(|1, 0\rangle + i|0, 1\rangle),$$

créant une superposition cohérente entre les deux chemins optiques.

5.3 Cas deux photons incident : interférence de Hong–Ou–Mandel

Si deux photons arrivent simultanément sur les deux entrées du BS :

$$|1, 1\rangle \rightarrow \frac{i}{\sqrt{2}}(|2, 0\rangle + |0, 2\rangle),$$

les photons sortent toujours ensemble. C'est le phénomène d'**interférence de Hong–Ou–Mandel**, signature du comportement quantique des photons.

6 Simulation dans Perceval

Le framework **Perceval** permet de construire ces circuits et de simuler leur évolution unitaire.

Exemple en Python

```
from perceval import *
from perceval.components import BS, PS
import math

# Circuit à 2 modes (1 qubit dual-rail)
c = Circuit(2)
c.add(0, BS(theta=math.pi/2)) # Beam splitter 50/50
c.add(0, PS(phi=math.pi/4))   # Phase shifter sur le mode 0
```

Perceval calcule automatiquement la matrice unitaire globale U , et permet :

- la simulation d'amplitudes de sortie,
- le *sampling* de détections,
- l'exécution sur un **QPU photonique** réel.

7 Résumé conceptuel

Élément	Signification
Mode	Voie optique pouvant contenir 0, 1, 2... photons
Base de Fock	États de nombre de photons $ n_1, \dots, n_m\rangle$
Encodage dual-rail	Qubit = un photon sur deux modes
Phase shifter (PS)	Multiplie un mode par $e^{i\phi}$
Beam splitter (BS)	Mélange deux modes via $U_{BS}(\theta)$
Opérateurs	$a^\dagger$ crée, a annihile un photon
Évolution	$a_{\text{out}}^\dagger = U a_{\text{in}}^\dagger$
Circuit photonique	Composition de PS et BS
Perceval	Framework Python pour la simulation photonique

8 Extension : Polarisation, Plaques Biréfringentes et Diviseurs Polarisants

8.1 Modes de polarisation et espace de Hilbert

Définition – Mode de polarisation

Outre le mode spatial, un photon possède un **degré de liberté de polarisation**, caractérisant la direction du champ électrique transverse. Ce degré de liberté est décrit dans la base orthonormée :

$$\{|H\rangle, |V\rangle\},$$

où $|H\rangle$ (horizontal) et $|V\rangle$ (vertical) sont les deux états de polarisation linéaire orthogonaux.

Chaque mode spatial k est donc scindé en deux sous-modes de polarisation :

$$k : H \quad \text{et} \quad k : V.$$

L'espace de Hilbert total s'écrit alors comme un produit tensoriel :

$$\mathcal{H} = \mathcal{H}_{\text{spatial}} \otimes \mathcal{H}_{\text{pol}}, \quad \mathcal{H}_{\text{pol}} = \text{span}\{|H\rangle, |V\rangle\}.$$

Un état général de polarisation d'un photon unique s'écrit :

$$|\psi\rangle = \alpha|H\rangle + \beta|V\rangle, \quad |\alpha|^2 + |\beta|^2 = 1.$$

Ce vecteur décrit la position du photon sur la **sphère de Poincaré**, analogue de la sphère de Bloch pour la polarisation.

8.2 Plaques biréfringentes : transformations unitaires sur la polarisation

Une **plaque biréfringente** (ou *waveplate*) introduit un retard de phase δ entre deux axes optiques — dits *rapide* et *lent* — orientés à un angle ξ par rapport à la base $\{|H\rangle, |V\rangle\}$. Elle agit comme une matrice unitaire 2×2 sur l'espace de polarisation $\mathcal{H}_{\text{pol}}$:

$$\hat{U}_{\text{WP}}(\delta, \xi) = \begin{pmatrix} i \sin(\delta) \cos(2\xi) + \cos(\delta) & i \sin(\delta) \sin(2\xi) \\ i \sin(\delta) \sin(2\xi) & -i \sin(\delta) \cos(2\xi) + \cos(\delta) \end{pmatrix}.$$

Interprétation des paramètres.

- δ : retard de phase entre les axes optiques rapide et lent (proportionnel à l'épaisseur et à la biréfringence du matériau). Il règle l'amplitude de la rotation sur la sphère de Poincaré.
- ξ : orientation des axes de la plaque par rapport à la base $\{H, V\}$; il détermine la direction de la rotation.

L'action d'une plaque biréfringente correspond à une **rotation unitaire sur la sphère de Poincaré**, et donc à une rotation de l'état de polarisation.

Cas particuliers fondamentaux

1. **Plaque demi-onde (HWP)** – retard $\delta = \pi/2$. Elle induit une rotation de l'état de polarisation d'un angle 2ξ :

$$\hat{U}_{\text{HWP}}(\xi) = \begin{pmatrix} \cos(2\xi) & \sin(2\xi) \\ \sin(2\xi) & -\cos(2\xi) \end{pmatrix}.$$

Pour $\xi = 45^\circ$, la HWP échange les composantes :

$$|H\rangle \mapsto |V\rangle, \quad |V\rangle \mapsto |H\rangle.$$

Elle agit ainsi comme un **miroir de polarisation**.

2. **Plaque quart-d'onde (QWP)** – retard $\delta = \pi/4$. Elle convertit les polarisations linéaires en polarisations circulaires ou elliptiques. Pour $\xi = 45^\circ$:

$$|H\rangle \mapsto \frac{|H\rangle + i|V\rangle}{\sqrt{2}} = |R\rangle, \quad |V\rangle \mapsto \frac{|V\rangle + i|H\rangle}{\sqrt{2}} = |L\rangle,$$

où $|R\rangle$ et $|L\rangle$ sont les polarisations circulaires droite et gauche.

Synthèse. Les HWP permettent de **faire tourner l'azimut** de la polarisation, tandis que les QWP assurent la conversion **linéaire** $\leftrightarrow$ **circulaire**. Les deux sont unitaires et conservent la norme du vecteur d'état.

8.3 Diviseur de faisceau polarisant (PBS)

Définition – Polarizing Beam Splitter (PBS)

Un **diviseur de faisceau polarisant** sépare les composantes de polarisation d'un photon : il *transmet* la composante horizontale $|H\rangle$ et *réfléchit* la composante verticale $|V\rangle$.

Le PBS agit simultanément sur la polarisation et sur les modes spatiaux, en redirigeant les photons selon leur polarisation. Dans la base à quatre états $\{|H, 0\rangle, |V, 0\rangle, |H, 1\rangle, |V, 1\rangle\}$ (deux chemins spatiaux), sa matrice unitaire idéale est :

$$\hat{U}_{\text{PBS}} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Lecture physique.

- $|H, 0\rangle \mapsto |H, 1\rangle$: la polarisation H est transmise vers l'autre sortie.
- $|V, 0\rangle \mapsto |V, 0\rangle$: la polarisation V reste dans le même bras.
- $|H, 1\rangle \mapsto |H, 0\rangle, |V, 1\rangle \mapsto |V, 1\rangle$: symétrie entre les deux voies.

Le PBS est donc une **permutation unitaire**, qui convertit l'information de polarisation en information spatiale — et réciproquement.

8.4 Applications et exemples

Exemple 1 – QWP à 45° : linéaire $\rightarrow$ circulaire.

$$|H\rangle \xrightarrow{\text{QWP}} \frac{|H\rangle + i|V\rangle}{\sqrt{2}} = |R\rangle, \quad |V\rangle \xrightarrow{\text{QWP}} \frac{|V\rangle + i|H\rangle}{\sqrt{2}} = |L\rangle.$$

Exemple 2 – HWP à $22,5^\circ$: rotation de 45° .

$$|H\rangle \mapsto \frac{|H\rangle + |V\rangle}{\sqrt{2}} = |D\rangle, \quad |V\rangle \mapsto \frac{|H\rangle - |V\rangle}{\sqrt{2}} = |A\rangle,$$

où $|D\rangle$ et $|A\rangle$ désignent les polarisations diagonales à $\pm 45^\circ$.

Exemple 3 – Conversion polarisation $\rightarrow$ spatial via PBS. Un photon dans l'état $|\psi\rangle = \alpha|H\rangle + \beta|V\rangle$ entrant sur le bras 0 devient après PBS :

$$|\psi, 0\rangle \mapsto \alpha|H, 1\rangle + \beta|V, 0\rangle.$$

Les composantes H et V sont séparées dans deux chemins distincts : la polarisation est ainsi convertie en information spatiale, utile pour interfacer les encodages *polarisation* et *dual-rail*.

8.5 Remarques expérimentales et modélisation

- **Phases et conventions** : selon les conventions optiques, certaines matrices comportent des facteurs $\pm i$. Ceux-ci n'affectent pas les probabilités mais influencent les interférences.
- **Unitarité globale** : les plaques agissent localement sur la polarisation (blocs 2×2), tandis qu'un PBS agit sur le produit $\mathcal{H}_{\text{spatial}} \otimes \mathcal{H}_{\text{pol}}$.
- **Bruits réalistes** : pertes dépendantes de la polarisation, diaphonie et fuite croisée peuvent être modélisées par des canaux non unitaires.

—

8.6 Implémentation dans Perceval

Dans le framework **Perceval**, la polarisation est représentée en **dupliquant chaque mode spatial** pour les composantes H et V . On y implémente :

- des WP paramétrées par (δ, ξ) agissant sur les paires (mode : H , mode : V) ;
- un PBS comme permutation sur les états $\{|H, 0\rangle, |V, 0\rangle, |H, 1\rangle, |V, 1\rangle\}$;
- les mêmes composants spatiaux (BS, PS) appliqués sur les bras résultants.

Ainsi, tout montage expérimental combinant QWP–HWP–PBS peut être décrit dans **Perceval** comme une suite d’unitaires locales et de permutations globales, appliquées à l’espace $\mathcal{H}_{\text{spatial}} \otimes \mathcal{H}_{\text{pol}}$.

8.7 Vérifications essentielles

1. Conservation de la norme : $\|\widehat{U}_{\text{WP}}|\psi\rangle\| = \||\psi\rangle\|$.
2. QWP à 45° : $|H\rangle \rightarrow |R\rangle, |V\rangle \rightarrow |L\rangle$.
3. HWP à 45° : $|H\rangle \leftrightarrow |V\rangle$.
4. PBS : séparation correcte des composantes H et V en chemins distincts.

9 Feedforward : contrôle adaptatif par la mesure

9.1 Limites des réseaux optiques linéaires et rôle du *feedforward*

Les photons sont d’excellentes particules pour porter l’information quantique : ils sont stables, rapides, et interagissent très faiblement avec leur environnement. Cependant, cette même faiblesse d’interaction rend le calcul quantique universel difficile à réaliser avec des réseaux purement optiques linéaires. ***En effet, les photons n’interagissent pratiquement pas entre eux, et sans interaction, aucune porte logique entanglante ne peut être directement implémentée.***

Chaque photon évolue alors indépendamment des autres : il n’existe aucune corrélation directe ni interaction entre eux. Autrement dit :

- Deux photons se croisant dans un *beam splitter* ne “se voient” pas.
- Leurs amplitudes interfèrent, mais ils ne se conditionnent pas mutuellement.

Cette linéarité rend les réseaux optiques élégants sur le plan mathématique, mais intrinsèquement incapables de produire les corrélations complexes nécessaires au calcul quantique universel.

9.1.1 Le besoin de non-linéarités pour le calcul universel

Pour être universel, un ordinateur quantique doit pouvoir réaliser à la fois :

- des rotations locales (unitaires à un seul qubit),
- et au moins une porte à deux qubits entanglante (comme la CNOT ou la CZ).

Ces dernières nécessitent une interaction entre qubits, absente dans un réseau optique linéaire. Aucune porte CNOT ou CZ ne peut donc être réalisée directement sans introduire un élément non-linéaire.

9.1.2 La mesure comme source de non-linéarité effective

Knill, Laflamme et Milburn (KLM, 2001) ont proposé une approche ingénieuse : utiliser la *mesure* comme source de non-linéarité. Lorsqu'un photon est mesuré, le système s'effondre dans un état particulier, un processus fondamentalement non-linéaire et non-déterministe. En conditionnant la suite du calcul au résultat de cette mesure, on obtient une *non-linéarité effective* :

La mesure joue le rôle d'une interaction logique entre photons, même s'ils ne se sont jamais rencontrés physiquement.

9.1.3 Principe du *feedforward*

Principe du feedforward (rétroaction en temps réel)

Le *feedforward* consiste à :

1. Mesurer un ou plusieurs photons auxiliaires (*ancillas*) ;
2. Utiliser le résultat de cette mesure pour modifier dynamiquement un paramètre du circuit (phase, bascule logique, etc.) ;
3. Poursuivre ensuite le calcul.

Ce mécanisme permet de :

- créer de l'intrication entre photons via des mesures conditionnelles,
- corriger le caractère aléatoire de la mesure afin de rendre le calcul quasi-déterministe.

9.1.4 Exemple : la porte CNOT photonique

Une version simplifiée de la porte CNOT en optique linéaire fonctionne ainsi :

- Deux photons (contrôle et cible) traversent un réseau de *beam splitters* et de *phase shifters*.
- Deux photons auxiliaires sont mesurés à la sortie.
- Si un certain motif de détection est observé (par exemple, un photon détecté dans le détecteur 1 et aucun dans le détecteur 2), la porte CNOT a “réussi”.
- Si le motif est différent, le calcul est annulé ou corrigé par *feedforward*.

Sans *feedforward*, la porte ne fonctionne que par post-sélection (succès aléatoire). Avec *feedforward*, elle devient quasi-déterministe, car le circuit s'adapte dynamiquement au résultat de la mesure.

9.1.5 Rôle de la mesure et du *feedforward*

Élément	Rôle
Réseau optique linéaire	Produit des interférences linéaires (aucune interaction réelle entre photons).
Mesure	Introduit une non-linéarité via l'effondrement du vecteur d'état.
<i>Feedforward</i>	Transforme cette non-linéarité aléatoire en opération conditionnelle et contrôlable.
Résultat final	Permet un calcul quantique universel sans interaction physique directe entre photons.

9.2 Implémentation dans Perceval

Perceval intègre nativement cette logique adaptative via trois types d'éléments :

1. Les **modes ancilla**, servant à la mesure et au contrôle ;
2. Les **détecteurs**, capables de “cliquer” ou non ;
3. Les **composants dépendants de la mesure**, dont les paramètres sont modifiés en fonction du résultat.

9.2.1 Exemple minimal en Python

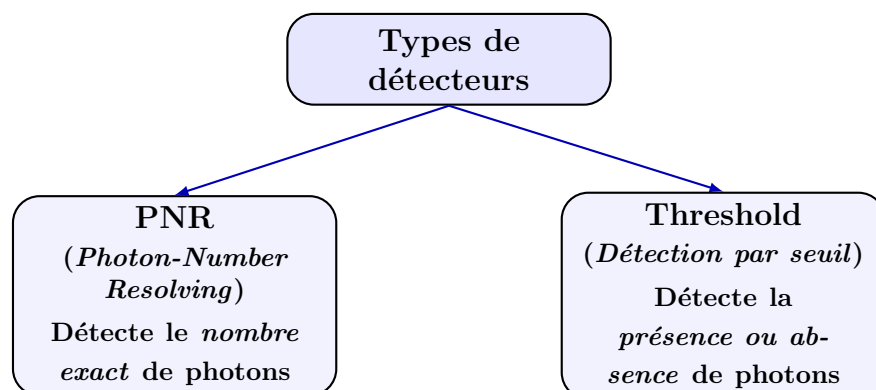
Exemple en Python

```
from perceval.components import Circuit, BS, PS, Detector
from perceval.utils import BasicState

# 1. Définition du circuit
c = Circuit(3) # 2 modes pour le qubit + 1 mode ancilla

# 2. Transformation optique linéaire
c.add(0, BS(theta=0.5)) # beam splitter entre modes 0 et 1
c.add(1, PS(phi=0.2))   # phase shifter sur le mode 1

# 3. Ajout du mode ancilla et détection
c.add(2, BS(theta=0.3)) # couplage avec l'ancilla
c.add(2, Detector())    # détection du photon ancilla
```



Détecteur PNR (Photon-Number Resolving)

```
from perceval.components import DetectorPNR

# Détecteur capable de distinguer le nombre exact de photons
c.add(2, DetectorPNR())
```

Détecteur Threshold

```
from perceval.components import DetectorThreshold

# Détecteur binaire : présence (click) ou absence de photons
c.add(2, DetectorThreshold())
```

9.2.2 Conditionnement du calcul

Le feedforward peut être implémenté dans Perceval de deux manières :

(a) **Post-sélection** On conserve uniquement les exécutions où la mesure correspond à un résultat spécifique :

```
c.with_postselection({2: 1}) # Conserve cas avec 1 photon détecté sur le mode 2
```

(b) **Logique conditionnelle explicite** Le résultat du détecteur modifie dynamiquement la suite du circuit :

```
def feedforward_action(measurement_result):
    if measurement_result[2] == 1:
        c.add(1, PS(phi=3.14)) # Correction de phase si détection
    else:
        c.add(1, PS(phi=0.0))  # Sinon aucune correction

c.add_feedforward_rule(feedforward_action)
```

Cette structure simule un circuit adaptatif : les composants optiques dépendent du résultat d'une mesure intermédiaire simulée dans Perceval.

9.3 Architecture interne du *feedforward* dans Perceval

Dans Perceval, chaque circuit optique est représenté comme un **graphe orienté** :

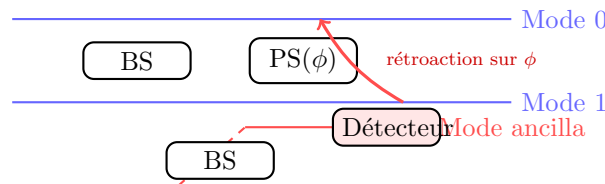
- les **nœuds** correspondent aux composants optiques (BS, PS, Detector, etc.) ;
- les **arêtes** représentent les modes optiques reliant ces composants.

Le *feedforward* est implémenté en ajoutant des **dépendances dynamiques** entre nœuds : le résultat d'un détecteur modifie les paramètres d'un composant ultérieur, permettant d'ajuster le circuit en temps réel.

Node₁ : $BS(\theta = 0.5)$ $\longrightarrow$ Node₂ : $\text{Detector}(\text{mode}=2)$ $\longrightarrow$ Node₃ : $PS(\phi = f(\text{Node}_2.\text{result}))$

Lors de la simulation, **Perceval** évalue les probabilités de détection de chaque motif, puis met à jour automatiquement les composants dépendants. Ce mécanisme reproduit un **calcul quantique adaptatif** — où le résultat d’une mesure influence directement la suite de l’évolution du système.

9.4 Schéma physique du feedforward (encodage dual-rail)



Lecture du schéma

- Le qubit est encodé en **dual-rail** : un photon réparti entre les modes 0 et 1 ;
- Un **mode ancilla** (en rouge) est couplé au circuit et mesuré ;
- Si le détecteur clique, la phase ϕ du **Phase Shifter** est ajustée en conséquence.

Ce montage illustre la structure typique d’un *feedforward photonique* : une mesure intermédiaire (sur un mode ancilla) modifie instantanément un paramètre optique, rendant le calcul quantique **adaptatif et conditionnel**.

9.5 Synthèse

Concept	Description
Principe physique	La mesure modifie dynamiquement un paramètre optique dans le circuit.
Objectif	Transformer un protocole probabiliste en protocole conditionnel quasi-déterministe.
Implémentation Perceval	Règles de dépendance entre détecteurs et composants adaptatifs (feedforward).
Usage typique	Portes multi-qubits, correction de phase, protocoles KLM.
Intérêt scientifique	Rend l’optique linéaire universelle pour le calcul quantique.

10 Structure interne de Perceval : du photon au processeur

L’objectif de cette section est de comprendre comment le framework **Perceval** modélise un calcul photonique linéaire, depuis la description des états quantiques jusqu’à la simulation d’un processeur optique complet.

Nous détaillerons successivement :

1. la représentation des **états quantiques** ;
2. la description des **circuits optiques** ;
3. les différents **backends de simulation** ;
4. et enfin la structure d'un **processeur photonique virtuel**.

10.1 Les États : BasicState et StateVector

Principe physique

Dans un circuit optique linéaire, l'information est encodée dans la **distribution des photons** sur plusieurs **modes**. Chaque mode correspond à une voie optique : un guide d'onde, une polarisation, ou une fréquence distincte.

Un état quantique de n photons dans m modes est décrit par un **état de Fock** :

$$|n_0, n_1, \dots, n_{m-1}\rangle$$

où n_i désigne le nombre de photons dans le mode i .

- $|1, 0\rangle$: un photon dans le premier mode ;
- $|1, 1\rangle$: un photon dans chaque mode ;
- $|2, 0\rangle$: deux photons indistinguables dans le même mode.

Représentation dans Perceval

Deux classes fondamentales permettent de manipuler ces états :

`BasicState` représente un état de Fock pur (déterministe) :

Exemple en Python

```
from perceval.utils import BasicState

s1 = BasicState("|1,0,0>") # 1 photon dans le mode 0
s2 = BasicState("|1,1,0>") # 1 photon dans les modes 0 et 1
```

Chaque photon peut posséder une **annotation**, indiquant par exemple sa polarisation ou son origine temporelle :

Exemple en Python

```
s3 = BasicState("|1_H,1_V>") # photon H (horizontal) et V (vertical)
```

Ces annotations servent à modéliser la **distinguabilité** entre photons, cruciale pour les phénomènes d'interférence.

`StateVector` représente une **superposition quantique** d'états de Fock :

$$|\psi\rangle = \alpha|1,0\rangle + \beta|0,1\rangle$$

Exemple en Python

```
from perceval.utils import StateVector, BasicState
import numpy as np

psi = StateVector({
    BasicState("|1,0>"): 1/np.sqrt(2),
    BasicState("|0,1>"): 1/np.sqrt(2)
})
```

Ainsi :

- `BasicState` $\Rightarrow$ état photonique pur,
- `StateVector` $\Rightarrow$ superposition cohérente d'états.

10.2 Les Circuits : Circuit et les composants optiques

Construction générale

La classe `Circuit` est le cœur de Perceval : elle assemble les composants optiques. Chaque composant agit sur un ou plusieurs modes et est décrit par une **matrice unitaire** U telle que :

$$\mathbf{a}_{\text{out}}^\dagger = U \mathbf{a}_{\text{in}}^\dagger.$$

L'unitaire globale du circuit s'obtient par composition :

$$U_{\text{global}} = U_N \cdots U_2 U_1.$$

Exemple pratique

Exemple en Python

```
from perceval.components import Circuit, BS, PS

c = Circuit(2)           # 2 modes
c.add(0, BS())           # beam splitter entre les modes 0 et 1
c.add(1, PS(phi=0.3))    # phase shifter sur le mode 1

U = c.compute_unitary() # matrice globale
```

On peut également construire un circuit à partir d'une matrice donnée (décomposition inverse) :

Exemple en Python

```
c.decompose(U)
```


Bibliothèque de composants intégrés

Composant	Matrice unitaire	Description
BS	2×2 rotation (θ, ϕ)	Mélange deux modes
PS	$e^{i\phi}$	Ajout de phase sur un mode
WP	rotation sur sous-espace H/V	Manipulation de polarisation
PBS	permutation 4×4	Séparation des polarisations
Permutation	échange de modes	Reconfiguration du réseau
TimeDelay	déphasage temporel	Retard optique

10.3 Les *Backends* : simulateurs de circuits photoniques

Principe général

La simulation d'un circuit photonique repose sur le calcul des **permanents** de sous-matrices unitaires, une opération réputée #P-difficile. **Perceval** fournit plusieurs **moteurs de calcul** (*backends*) adaptés à différents contextes expérimentaux et niveaux de précision.

1. CliffordClifford2017

Implémente l'algorithme de **Clifford & Clifford (2017)**. Il permet la **génération d'échantillons de sortie** selon :

$$\mathcal{O}(n2^n + mn^2)$$

Ce backend est particulièrement efficace pour les expériences de **Boson Sampling**, où seules les configurations échantillonnées sont nécessaires.

2. Naive

Calcule directement les **permanents** via les algorithmes classiques de **Ryser** ou **Glynn**. Il offre une simulation exacte, mais sa complexité est exponentielle :

$$\mathcal{O}(n!)$$

Convient aux circuits de petite taille ou aux vérifications analytiques de précision.

3. SLOS (Strong Linear Optical Simulation)

Ce backend calcule la **distribution complète des sorties** :

$$\mathcal{O}\left(n \binom{m+n-1}{n}\right)$$

Très performant jusqu'à environ une vingtaine de photons, il est idéal pour explorer toutes les probabilités de détection possibles dans un réseau linéaire.

4. Stepper

Le **Stepper** applique chaque composant séquentiellement, sans calculer la matrice globale U . Il est conçu pour les **circuits non unitaires**, incluant bruit, pertes ou éléments adaptatifs :

Complexité : linéaire en nombre de composants.

Approprié pour le **débogage** et les simulations expérimentales réalistes.

10.4 Les Processeurs : Processor

Concept

Un Processor regroupe dans une même entité :

1. les **sources de photons** ;
2. le **circuit optique** ;
3. les **détecteurs** de sortie.

C'est donc l'abstraction d'un **processeur photonique complet**, comparable à un dispositif expérimental réel .

Exemple

Exemple en Python

```
from perceval import Processor, BasicState
from perceval.components import Circuit, BS, PS

# Circuit
c = Circuit(2)
c.add(0, BS(theta=0.5))
c.add(1, PS(phi=0.3))

# Processeur
p = Processor("SLOS", c)
p.with_input(BasicState("|1,1>"))
result = p.probs()
```

Le processeur prend en compte :

- les imperfections des sources (pertes, pureté, indiscernabilité) ;
- les caractéristiques du circuit (unitaires ou dissipatifs) ;
- la nature des détecteurs (PNR, seuil) ;
- le post-traitement (post-sélection, feedforward).

10.5 Vision d'ensemble

Niveau	Objet Perceval	Rôle	Équivalent physique
1	BasicState	État de Fock	Configuration photonique
2	StateVector	Superposition d'états	État quantique complet
3	Circuit	Composants optiques	Réseau d'interférences
4	Backend	Algorithme de simulation	Moteur de calcul
5	Processor	Source + circuit + détecteurs	Expérience photonique complète

Calcul des probabilités avec le backend SLOS

```
# --- Calcul des probabilités de sortie ---
result = p.probs()

print(result)
# Exemple de sortie :
# {
#     "|2,0>": 0.45,
#     "|1,1>": 0.10,
#     "|0,2>": 0.45
# }
```

Le processeur simule toutes les configurations de sortie possibles après passage dans le réseau optique. Chaque valeur représente la probabilité d'obtenir une distribution donnée de photons, notée $|n_0, n_1\rangle$.

Ces probabilités sont calculées à partir des **permanents** de sous-matrices de la matrice unitaire U associée au circuit, par le backend choisi (ici "SLOS").

10.6 Exécution réelle sur QPU (Quandela Cloud)

Perceval ne se limite pas à la simulation : il permet également de soumettre le circuit à un **processeur quantique photonique réel** via l'infrastructure Quandela Cloud.

Exécution sur le QPU photonic réel

```
from perceval.runtime import RemoteProcessor
from perceval import BasicState

# --- Définition du processeur distant ---
p = RemoteProcessor("Quandela_QPU")

# --- Chargement du circuit et de l'état d'entrée ---
p.with_input(BasicState("|1,1>"))
p.with_circuit(c)

# --- Paramètres d'exécution ---
p.sample_count = 1000 # Nombre de tirs (shots)
results = p.run()

print(results)
```

Description du fonctionnement

- `RemoteProcessor` établit une connexion sécurisée avec un processeur photonique réel, hébergé dans l'infrastructure **Quandela Cloud**.
- L'utilisateur définit :
 - le **circuit optique** à exécuter ;
 - l'**état d'entrée** (distribution des photons injectés) ;
 - le nombre d'échantillons à collecter (**sample_count**).
- Le processeur exécute physiquement le calcul quantique en laboratoire, en envoyant de véritables photons dans un circuit intégré, et renvoie les **statistiques expérimentales mesurées**.

Résultat obtenu

Le résultat est un ensemble de comptages (ou *shots*) correspondant à chaque configuration de sortie observée, par exemple :

$|2, 0\rangle$: 452 détections
 $|1, 1\rangle$: 102 détections
 $|0, 2\rangle$: 446 détections

Ces données expérimentales peuvent ensuite être normalisées pour obtenir les probabilités empiriques, comparables aux prédictions théoriques du simulateur.

10.7 Résumé : du simulateur au QPU réel

Niveau	Outil Perceval	Type d'exécution	Objectif principal
1	Backend (SL0S, Naive, Stepper)	Simulation mathématique	Probabilités exactes ou éc
2	Processor	Simulation expérimentale	Modélisation réaliste (sou
3	RemoteProcessor	Exécution sur QPU réel	Calcul quantique photon

Ainsi, le même circuit `Perceval` peut être :

- simulé numériquement pour l'analyse théorique ;
- exécuté virtuellement avec bruit et pertes ;
- ou envoyé directement sur un processeur photonique réel, pour obtenir des résultats issus de photons véritablement manipulés.

11 Architecture du QPU photonique

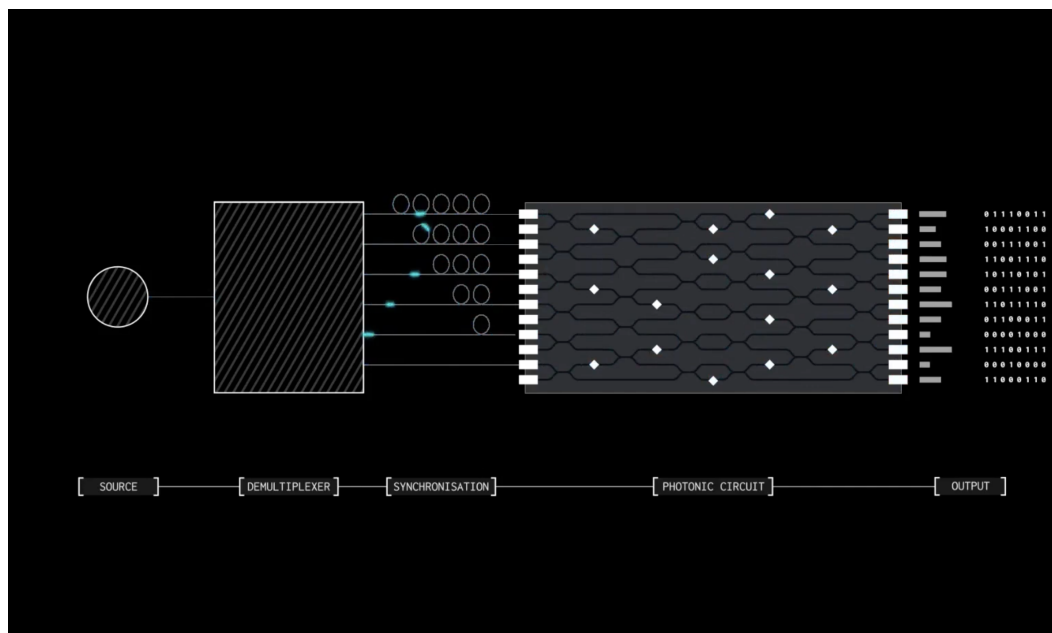


Figure 1: Illustration du QPU photonique de Quandela : de la source de photons uniques à la puce intégrée. *Image extraite du Webinar Champion 2024 – Quandela, présenté le 14 mars 2025 par Pierre-Emmanuel Émériaux (Head of Algorithms Team) et Ariane Soret (Quantum Information Scientist, Quandela).*

11.0.1 (a) Source de photons uniques — Single Photon Source (SPS)

Le cœur du QPU est une **source de photons uniques (SPS)**. Il s'agit d'une *quantum dot* (boîte quantique) excitée par un laser, qui émet un seul photon à chaque impulsion, avec toujours les mêmes caractéristiques (fréquence, polarisation, phase).

Chaque excitation génère un photon identique : les émissions successives forment un **train temporel de photons uniques**. Ces photons constituent la base du calcul quantique photonique.

11.0.2 (b) Le démultiplexeur temporel

Ces photons sont ensuite envoyés dans un **démultiplexeur temporel**, qui convertit la séquence temporelle en **distribution spatiale**. On obtient ainsi plusieurs photons présents simultanément sur des modes optiques distincts — typiquement jusqu'à six qubits encodés en trajectoire (*dual-rail encoding*).

Défi expérimental : les photons doivent arriver exactement en même temps sur le processeur. Des *lignes à retard* en fibre optique assurent cette synchronisation, mais elles sont sensibles à la température, ce qui nécessite une compensation active.

Résumé

Des photons émis successivement sont redirigés dans des chemins différents, permettant d'obtenir plusieurs photons simultanément dans le circuit à partir d'une seule source.

11.0.3 (c) La puce photonique intégrée

Les photons synchronisés pénètrent ensuite dans une **puce photonique intégrée** — un réseau de guides d'onde, de déphaseurs et de diviseurs de faisceau.

Cette puce implémente les transformations unitaires du calcul quantique. Chaque composant correspond à une opération physique :

- un **déphaseur** applique une rotation de phase (R_z) ;
- un **beam splitter (BS)** crée une superposition entre deux chemins (analogue d'une porte Hadamard) ;
- un couple **BS + phase shifters** forme un **interféromètre de Mach–Zehnder (MZI)** capable de réaliser toute opération $SU(2)$.

En contrôlant les tensions appliquées à 126 **phase shifters thermo-optiques**, on reconfigure dynamiquement la puce.

Défi : les phase shifters ont une *diaphonie thermique* : chauffer un élément perturbe les voisins. Il faut donc un modèle précis pour compenser ces interférences thermiques.

À la sortie, les photons sont détectés, et les **corrélations entre détecteurs** révèlent les amplitudes de probabilité :

$$P(x) = |\langle x | \psi \rangle|^2$$

exactement comme dans le formalisme de la mécanique quantique.

11.0.4 (d) Vers l'intrication photonique

La puce actuelle, accessible via le **Quandela Cloud**, permet de manipuler jusqu'à **12 qubits photoniques**. Les futures générations viseront l'injection de **photons intriqués**, permettant la création de portes multi-qubits et l'extension de la puissance du QPU.

11.1 Algorithmes photoniques implémentés

Quandela développe également une suite d'algorithmes exploitant spécifiquement la physique des photons.

- **Variational Quantum Eigensolver (VQE)** Estimation variationnelle de la plus basse valeur propre d'un Hamiltonien. L'architecture photonique, naturellement linéaire, permet d'implémenter efficacement les circuits variationnels. *Applications : chimie quantique, modélisation moléculaire, optimisation énergétique.*
- **Quantum Random Number Generator (QRNG)** Exploite l'indéterminisme fondamental d'un photon sur un séparateur de faisceau. *Applications : cryptographie, sécurité, simulation stochastique.*
- **QUBO — Quadratic Unconstrained Binary Optimization** Optimisation quadratique discrète $f(x) = x^T Q x$. Les interférences lumineuses encodent directement les interactions entre variables binaires. *Applications : logistique, finance, conception moléculaire.*
- **Graph Problems** De nombreux problèmes de graphes se ramènent au calcul de **permanentes de matrices**. Or, un réseau optique linéaire réalise naturellement une permanente par interférence multi-photonique. *Applications : chimie structurale, détection de motifs, optimisation combinatoire.*

- **Quantum Machine Learning (QML)** Les données sont encodées dans les phases ou amplitudes optiques. Le réseau photonique agit comme une couche de neurones optiques. *Applications : classification, reconnaissance de formes, détection d'anomalies.*

11.2 Avantages structurels de la photonique

11.2.1 1. Fabrication industrielle

Les circuits photoniques sont fabriqués en salle blanche, comme les composants optoélectroniques classiques. Pas de cryogénie ni de vide extrême : ils fonctionnent à température ambiante.

Idée clé

La photonique rend possible le **scaling industriel** : la production de QPU en série devient envisageable.

11.2.2 2. Compatibilité avec le HPC

Les QPU photoniques fonctionnent sans refroidissement cryogénique, et peuvent donc être intégrés directement dans les centres de calcul classiques. **Avantage** : faible consommation énergétique et stabilité environnementale.

11.2.3 3. Hybridation naturelle

Les photons circulent à la vitesse de la lumière dans les fibres optiques. Ils s'intègrent donc naturellement dans les architectures hybrides quantique-classique :

$$\text{Photonique} \longleftrightarrow \text{HPC}$$

Conséquence : une intégration fluide entre QPU et supercalculateurs, sans barrière physique.

11.2.4 4. Flux de calcul photonique

Contrairement aux architectures à qubits statiques (supraconducteurs, ions piégés), le calcul photonique est **dynamique** : les qubits (photons) circulent dans le circuit. Chaque composant agit sur eux comme une étape d'un pipeline optique.

Avantage : cela ouvre la voie à un *calcul quantique en flux continu*, voire à un **réseau de processeurs quantiques interconnectés par fibre optique**.

Vision prospective

La photonique est la seule plateforme naturellement distribuée : elle transporte l'information quantique à grande distance. C'est la base de l'**Internet quantique**.

11.3 Conclusion : de la recherche à l'industrie

L'approche de Quandela illustre une continuité rare entre **mathématiques fondamentales, recherche expérimentale et déploiement industriel**. Le photon, longtemps objet d'étude de la mécanique quantique, devient ici une *brique technologique* de calcul et de communication quantique.

Perceval, la plateforme logicielle open source de Quandela, incarne cette transition : elle permet de passer du **code Python** à l'**expérience photonique réelle**, sur une même pile logicielle.

Références et sources

- **Quandela**, *Perceval: A Software Platform for Discrete Variable Photonic Quantum Computing*, Quantum 7, 1210 (2023). DOI: 10.22331/q-2023-02-07-1210.
- **Quandela**, *One nine availability of a Photonic Quantum Computer on the Cloud toward HPC integration*, 2024. White paper disponible via le Quandela Cloud : <https://cloud.quandela.com/>.
- **Perceval Documentation**, version 1.0 — *Official Developer Documentation*, Quandela (2025). Disponible en ligne : <https://perceval.quandela.net/docs/v1.0/index.html>.